LaTeX Author Guidelines for CVPR Proceedings

Anonymous CVPR submission

Paper ID *****

Abstract

The ABSTRACT is to be in fully justified italicized text, at the top of the left-hand column, below the author and affiliation information. Use the word "Abstract" as the title, in 12-point Times, boldface type, centered relative to the column, initially capitalized. The abstract is to be in 10-point, single-spaced type. Leave two blank lines after the Abstract, then begin the main text. Look at previous CVPRabstracts to get a feel for style and length.

1. Introduction

The rapid advancement of large language models (LLMs) has significantly expanded the capabilities of autonomous agents in reasoning, planning, and tool use. Building on these capabilities, LLM-based multi-agent systems (LLM-MAS) organize multiple agents to decompose complex objectives, exchange information, and coordinate actions. By further grounding such agents in physical or simulated environments, Embodied Multi-Agent Systems (EMAS) enable multiple agents to jointly perceive, communicate, and act over long-horizon tasks. Recent progress in EMAS has demonstrated increasingly sophisticated collaboration in navigation, household activities, manipulation, and social interaction.

Despite these advances, real-world embodied execution is prone to faults, which differ in *when* they lead to failure. **Manifest Faults**, such as agent crashes, dropout. In contrast, **Latent Execution Faults** are execution-time abnormalities, *e.g.*, in perception, memory, planning, communication, or coordination, that leave all agents operational and the team topology intact, yet cause the actual execution state to diverge from the state believed by the agents. Raising no immediate signal, a local deviation may be accepted by other agents, propagated through communication, and incorporated into subsequent plans, until the accumulated divergence surfaces as a system-level failure many steps later. This delay makes latent faults hard to notice and attribute, as the root cause may lie far back in the trajectory and in a different agent, but it also opens a window for detection and correction before the fault becomes fatal.

Current studies provide limited means to study such failures. Most EMAS are evaluated under nominal simulation conditions, where discrepancies between intended and actual execution are limited, and changes in world states or team configurations are anticipated by the task design rather than caused by execution faults. Existing robustness studies mostly focus on manifest structural changes such as agent removal, abstract LLM-level perturbations, or isolated embodied failures, and rarely distinguish faults by when their consequences emerge. As a result, current evaluations reveal little about whether agents can detect latent deviations before they lead to failure, localize their source, or recover before errors propagate across the team. This leaves a fault-related sim-to-real gap in current EMAS evaluation pipelines.

To address this gap, we introduce a **Fault Injection Layer (FIL)** for systematic reliability evaluation of EMAS, together with a **fault-aware embodied multi-agent framework** for closed-loop recovery. FIL operates over existing embodied environments without changing their task goals, dynamics, or native interfaces, and injects both manifest and latent faults grounded in key execution stages. Each fault is parameterized by its type, target, onset time, duration, severity, and ground-truth state, enabling controllable and traceable trajectories from fault onset to failure. The framework handles manifest faults through liveness monitoring and team reconfiguration, and focuses on latent faults: using only ordinary execution evidence without privileged fault labels, it detects them, localizes their likely sources, and performs diagnosis-conditioned recovery before they escalate into failure.

We deploy FIL and the fault-aware framework in Habitat-MAS, PARTNR, and VirtualHome-Social, spanning navigation, long-horizon household collaboration, and interactive social cooperation. Beyond final task success, we evaluate fault propagation, time to failure, localization quality, recovery effectiveness, and recovery overhead, enabling direct comparison between systems that merely tolerate faults and those that identify and correct their underlying causes before failure occurs.

The main contributions are summarized as follows:

- We distinguish **Manifest Faults**, which fail at onset, from **Latent Execution Faults**, which fail with a delay, and identify the latter as an under-explored yet deployment-relevant failure model for EMAS. We develop a cross-environment **Fault Injection Layer** for controllable, reproducible, and traceable injection of both classes.
- We develop a fault-aware embodied multi-agent framework that detects latent faults from ordinary execution traces, localizes their likely sources, and performs diagnosis-conditioned recovery before they escalate into system failure.
- We establish a cross-environment evaluation methodology spanning Habitat-MAS, PARTNR, and VirtualHome-Social that jointly measures task outcomes, fault propagation, time to failure, localization quality, recovery effectiveness, and recovery overhead.

2. Related Work

3. Methodology

We first formalize latent execution faults in EMAS (Sec. 3.1) and organize them into a two-level taxonomy (Sec. 3.2). We then present the Fault Injection Layer (FIL) that instantiates this taxonomy over existing embodied environments (Sec. 3.3), and the fault-aware framework that detects, localizes, and recovers from these faults online (Sec. 3.4). Finally, we describe the evaluation protocol.

3.1. Problem Formulation

Embodied multi-agent execution. We consider a team of N embodied agents $\mathcal{A} = \{a_1, \ldots, a_N\}$ acting in an environment with true state s_t at step t, on a task specified by goal predicates G. Execution relies on two levels. At the team level, a configuration $\mathcal{C}_t$ records which agents are active and responsive and the identity and role bound to each. At the agent level, each agent a_i has a capability set Σ^i of skills, tools, and physical reach, and acts not on s_t but on a belief state b_t^i about the world, the team, and the task progress.

Within each agent, information flows through a set of functional components. Since EMAS architectures vary widely (*e.g.*, centralized *vs*. decentralized planning, explicit *vs*. implicit memory, symbolic *vs*. low-level control), we group these components by role rather than by module:

- *Belief formation*: perception maps an observation o_t^i to a percept z_t^i, memory retrieves a decision context c_t^i from its store M_t^i, and communication delivers messages $m_t^{j\to i}$ from teammates.
- *Decision*: planning produces a plan π_t^i of ordered subgoals, and coordination allocates subtasks and shared resources across agents.
- *Execution and feedback*: action issues a skill call u_t^i from Σ^i and receives a status r_t^i, and verification judges (sub)task completion as v_t^i.

A system may merge, share (*e.g.*, a centralized planner), or omit components, so the decomposition serves only as a reference frame for locating faults and placing injection points (Sec. 3.3). We denote the set of components by $\mathcal{K}$ and the operator of component k for agent a_i by F_k^i. A fault can thus corrupt either $\mathcal{C}_t$, disrupting collaboration at onset, or the information flowing through the components of running agents, silently distorting their beliefs. These two cases are the manifest and latent faults defined next.

Latent execution faults. Let $\phi(s_t)$ project the true state onto the belief space, and let $\Delta_t^i = d\big(b_t^i, \phi(s_t)\big)$ be the divergence between what a_i believes and what is actually the case. A fault f perturbs a stage operator F_k^i over a time interval. We call f a *latent execution fault* if (i) all agents remain alive and the team topology is preserved, so that f is not exposed by structural signals such as liveness or membership, and (ii) it induces a divergence $\Delta_t^i > 0$ that the affected agent does not recognize as erroneous. Because a faulty output is formatted identically to a valid one, it can be consumed by downstream stages of the same agent and, through communication and coordination, by other agents.

Fault propagation. We model information flow as a directed provenance graph whose nodes are information items (percepts, memory entries, messages, plans, assignments, execution statuses, and verification results) and whose edges link each item to the items it was derived from. An item is *contaminated* if it is the output of an injected fault or is derived from a contaminated item; an agent is contaminated at step t if a decision it makes at t consumes a contaminated item. Latent faults are hazardous because contamination can spread silently across stages and agents before the task visibly fails.

Objectives. This formulation leads to two problems: (1) Generating latent execution faults in a controllable, reproducible, and traceable manner. (2) Enabling the team to detect, localize, and recover from them online using only ordinary execution evidence, *i.e.*, without access to the injected ground truth.

3.2. Fault Taxonomy

We organize execution faults into a two-level taxonomy (Tab. 1). Level-1 (L1) categories correspond to the execution stage at which erroneous information first enters the system, and Level-2 (L2) instances specify its concrete manifestation. Faults are thus classified by their *effect* rather than their physical *cause*. For example, occlusion is not a separate category: an occlusion that causes an object to be

mislocalized, missed, or placed in a wrong spatial relation is recorded as the corresponding perception instance. This keeps categories mutually exclusive and aligns each fault with the stage at which it must eventually be diagnosed and repaired. The taxonomy comprises nine L1 categories and 33 L2 instances. Structure faults (*e.g.*, agent dropout) are partially observable at the system level; we include them for completeness and as a contrast class, while the remaining eight categories are latent by construction and form the focus of this work.

3.3. Fault Injection Layer

3.4. Fault-Aware Embodied Multi-Agent Framework

References

Table 1. Two-level taxonomy of execution faults in EMAS. L1 categories indicate the execution stage at which erroneous information first enters the system, while L2 instances specify its concrete manifestation. Structure faults serve as a partially visible contrast class; all remaining categories are latent.

Category (L1)	**Instances (L2)**			
	ID	**Instance**	**ID**	**Instance**
VISIBLE · CONTRAST CLASS				
Structure Fault	ST-1	Agent Dropout	ST-2	Agent Responsiveness Degradation
	ST-3	Agent Configuration Mismatch		
LATENT · INDIVIDUAL COGNITION				
Perception Fault	PF-1	Object Localization Error	PF-2	Object Recognition Error
	PF-3	Spatial Relation Error	PF-4	Agent Perception Error
Memory Fault	MF-1	Memory Integrity Error	MF-2	Memory Retrieval Error
	MF-3	Context Distortion		
Planning Fault	PL-1	Goal Formulation Error	PL-2	Subgoal Omission
	PL-3	Plan Dependency Violation	PL-4	Planning Stagnation
LATENT · TEAM INTERACTION				
Communication Fault	CF-1	Message Delivery Error	CF-2	Message Addressing Error
	CF-3	Referential Error	CF-4	Message Validity Error
Coordination Fault	CO-1	Task Assignment Error	CO-2	Resource Conflict
	CO-3	Coordination Stagnation	CO-4	Collaboration Protocol Violation
LATENT · EMBODIED EXECUTION				
Capability Fault	CA-1	Capability Mismatch	CA-2	Tool Suitability Error
	CA-3	Tool Timeout	CA-4	Reachability Violation
Action Fault	AF-1	Action No-op	AF-2	Action Mismatch
	AF-3	Execution Deviation	AF-4	Motion Failure
	AF-5	Execution Reporting Error		
Verification Fault	VF-1	Verification Coverage Error	VF-2	Verification Misjudgment
	VF-3	Cross-agent Verification Failure		